

EDGE / CONTOUR-BASED OBJECT COUNTER & SORTER

*A Classical Computer Vision Pipeline for Automated
Object Detection, Counting, and Shape/Size Sorting*

Project Report

Submitted for CSE3010 — Computer Vision (VITYarthi – Build Your Own Project)

Field	Details
Course Code	CSE3010
Course Title	Computer Vision
Project Title	Edge/Contour-Based Object Counter & Sorter
Student Name	[Your Name]
Registration Number	[Your Registration Number]
Program / School	[Your Program], [Your School]
Faculty In-Charge	Dr. Soundarrajan
Submission Date	September 2026

2. Introduction

Computer Vision enables machines to interpret and act upon visual information the way humans do, and one of its most immediately useful applications is automated object counting and sorting. Tasks such as counting coins, sorting manufactured components by size, or tallying items on a conveyor belt are traditionally performed manually, which is slow, tedious, and prone to human error.

This project implements a lightweight, fully classical (non-machine-learning) Computer Vision pipeline that detects, counts, and sorts objects present in a static image. The system is built directly on the foundational techniques covered in the CSE3010 syllabus: Module 1 (image enhancement, histogram processing, and convolution/filtering) and Module 3 (edge detection using Canny, and edge-based image segmentation). Because the entire decision-making process relies on geometric descriptors — contour area, perimeter, circularity, and polygon vertex count — rather than a trained model, the system is fast, fully deterministic, does not need labeled training data, and every decision it makes can be explained and audited.

The report that follows documents the problem being solved, the functional and non-functional requirements, the system's architecture and design (with UML-style diagrams), key implementation details, the results obtained on two representative sample images, the automated testing strategy, the challenges encountered during development, and directions for future enhancement.

3. Problem Statement

Manually counting and sorting objects in an image — coins on a table, components on a conveyor belt, cells under a microscope, or fruits on a sorting line — is slow, error-prone, and does not scale with volume. Existing deep-learning object-detection solutions are powerful but are comparatively heavyweight: they require labeled training data, GPU resources, and considerable engineering effort, which is disproportionate for simple, well-controlled scenes where objects are clearly distinguishable from a plain background.

This project addresses that gap by building a lightweight, classical Computer Vision pipeline that automatically detects, counts, and sorts objects in a static image using edge detection and contour analysis — without needing a trained model, GPU, or labeled dataset. The pipeline must reliably distinguish individual objects from background/noise, describe each object geometrically, classify it by shape and size, and present the results in both a visual (annotated image) and tabular (CSV report) form.

4. Functional Requirements

The system is organized into three major functional modules, each with a clear input/output contract, connected in a single linear pipeline that the user triggers via the command line.

Module 1 — Image Pre-processing

- Input: a raw image file (PNG/JPG/BMP/TIFF) supplied via the `--image` CLI argument.
- Processing: grayscale conversion → CLAHE histogram equalization → Gaussian blur.
- Output: a cleaned, noise-suppressed grayscale image ready for edge detection.

Module 2 — Detection & Segmentation

- Input: the pre-processed grayscale image.
- Processing: Canny edge detection → morphological closing/dilation → external contour extraction → area-based noise filtering → geometric descriptor computation (area, perimeter, bounding box, centroid, polygon-approximation vertex count, circularity).
- Output: a list of DetectedObject records, one per surviving object.

Module 3 — Classification & Reporting

- Input: the list of DetectedObject records and the original image.
- Processing: shape classification (circle / triangle / square / rectangle / polygon) and size classification (small / medium / large); annotation of the original image with contours, bounding boxes, IDs, and labels; CSV export; aggregate summary statistics.
- Output: an annotated image, a per-object CSV report, and a console/log summary.

Logical Workflow

The user runs main.py with an image path. The CLI validates the input, then sequentially invokes Module 1, Module 2, and Module 3, finally writing all artifacts to the specified output directory and printing a summary to the console. The complete step-by-step flow is shown in the Workflow Diagram (Section 7.2).

5. Non-Functional Requirements

Requirement	How it is addressed
Performance	The full pipeline (pre-processing → detection → classification → reporting) completes in well under 100 ms for a typical 640×480–700×500 image on a standard CPU, since no model inference is involved.
Reliability	Deterministic, rule-based logic (no random initialization) means the same input always produces the same output; 25 automated unit tests guard against regressions.
Usability	A simple CLI (--image, --output, --min-area, --canny-low, --canny-high) with sensible defaults; clear console summary; annotated image is self-explanatory.
Maintainability	Strict modular separation (config / preprocessing / edge_detection / segmentation / shape_classifier / annotator / reporting / validation); every tunable threshold lives in one PipelineConfig dataclass.
Error Handling Strategy	Centralized validation.py raises typed exceptions (InvalidInputError, ImageLoadError) with actionable messages; main.py catches and reports them cleanly instead of crashing with a raw traceback.
Logging / Monitoring	Every module logs through a shared logger (logger_utils.py) to both console and output/pipeline.log, capturing each pipeline stage, timing, and any errors.
Resource Efficiency	Pure NumPy/OpenCV array operations; no external network calls or large model weights; peak memory stays proportional to image size.

Requirement	How it is addressed
Scalability	The pipeline is stateless per image, so batch-processing many images (e.g., a folder) can trivially be parallelized across processes in a future extension.

6. System Architecture

The system follows a layered, pipeline-oriented architecture. A thin CLI layer accepts and validates user input, three functional-module layers perform the actual computer-vision processing, and cross-cutting concerns (configuration and logging) are available to every layer. The diagram below shows the full architecture and the direction of data flow.

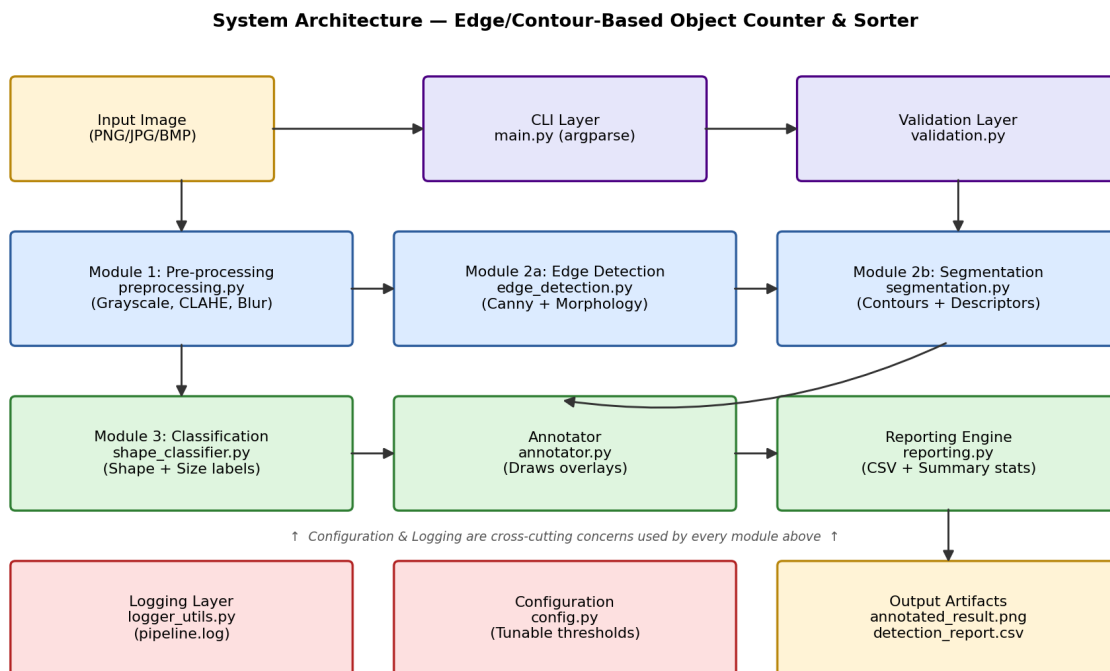


Figure 6.1 — System Architecture Diagram

Data flows strictly left-to-right, top-to-bottom: the input image is validated and handed to Module 1 (Pre-processing), whose output feeds Module 2a (Edge Detection) and Module 2b (Segmentation), whose output feeds Module 3 (Classification), which in turn drives the Annotator and Reporting Engine to produce the final output artifacts. Configuration and Logging are consulted by every module but do not sit in the main data-flow path.

7. Design Diagrams

7.1 Use Case Diagram

The system has a single actor — the User/Student running the tool — who can trigger seven distinct use cases, all of which are provided by the Object Counter & Sorter System.

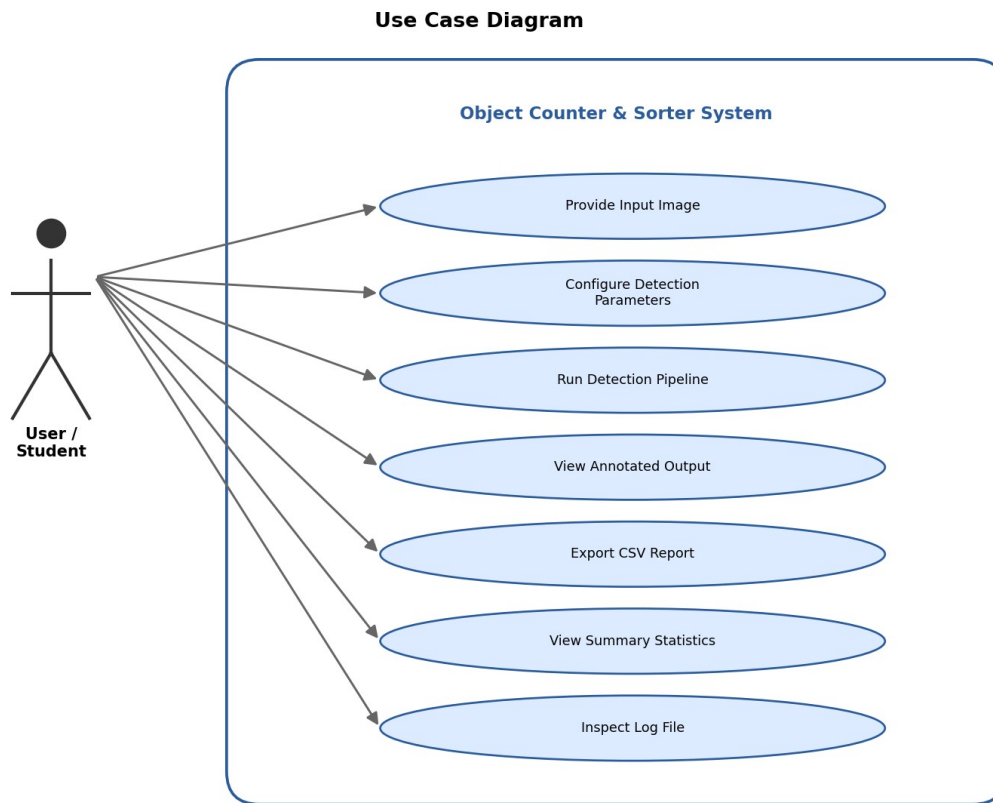


Figure 7.1 — Use Case Diagram

7.2 Workflow / Process Flow Diagram

The process flow diagram traces every step the pipeline performs on a single image, from CLI invocation to the final saved artifacts.

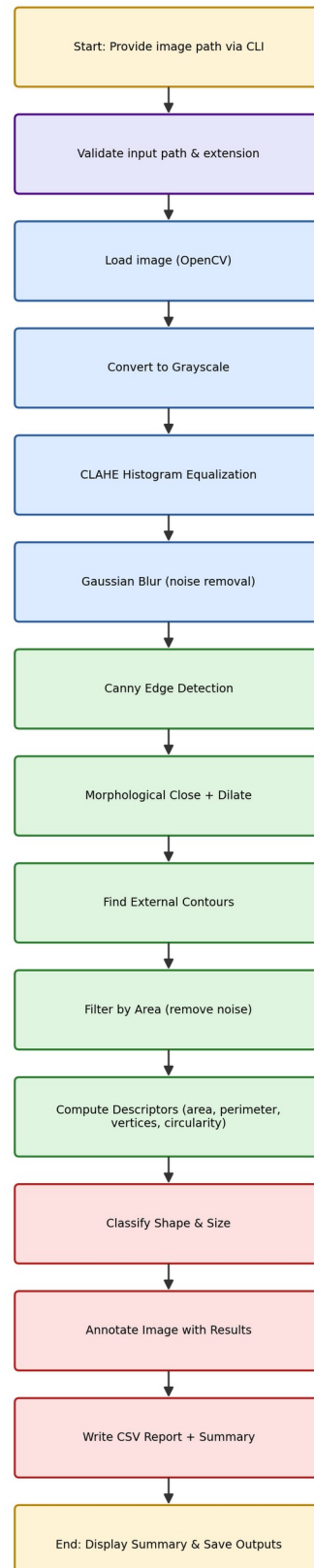
Process Flow

Figure 7.2 — Process Flow Diagram

7.3 Sequence Diagram

The sequence diagram below shows the exact order of method calls between the CLI orchestrator (main.py) and each functional module during a single end-to-end run.

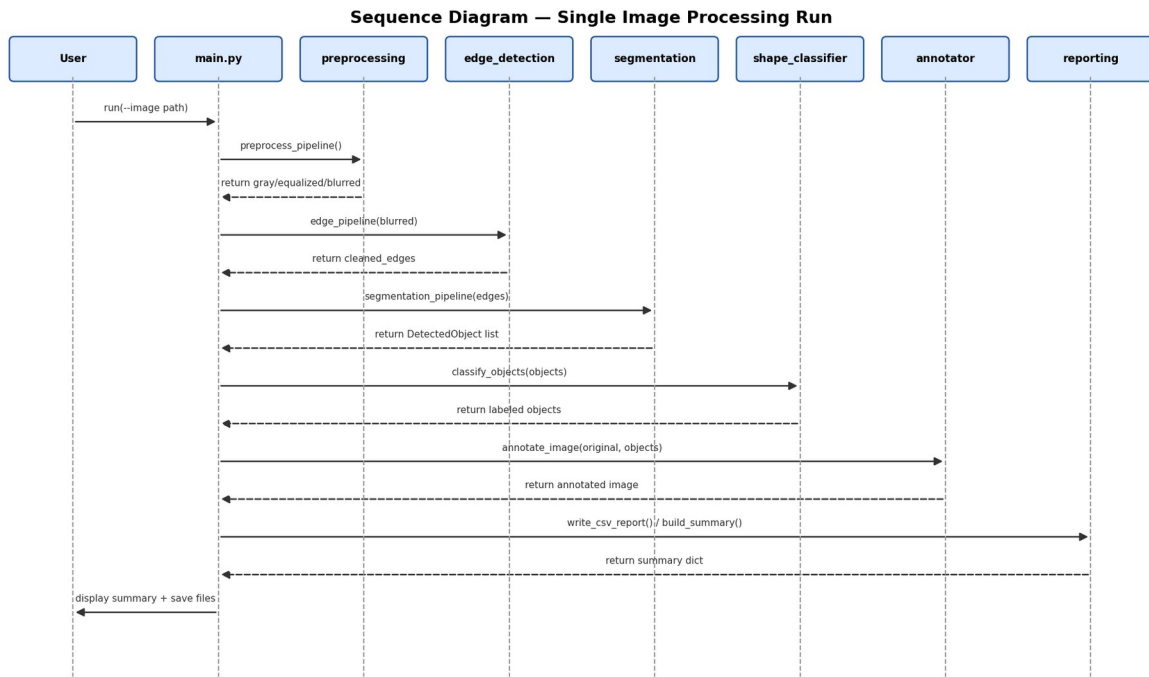


Figure 7.3 — Sequence Diagram for a Single Image Processing Run

7.4 Class / Component Diagram

The class/component diagram shows the two core data classes (PipelineConfig, DetectedObject), the custom exception types, the logger utility, and the eight functional/support modules, along with their key public functions and relationships.

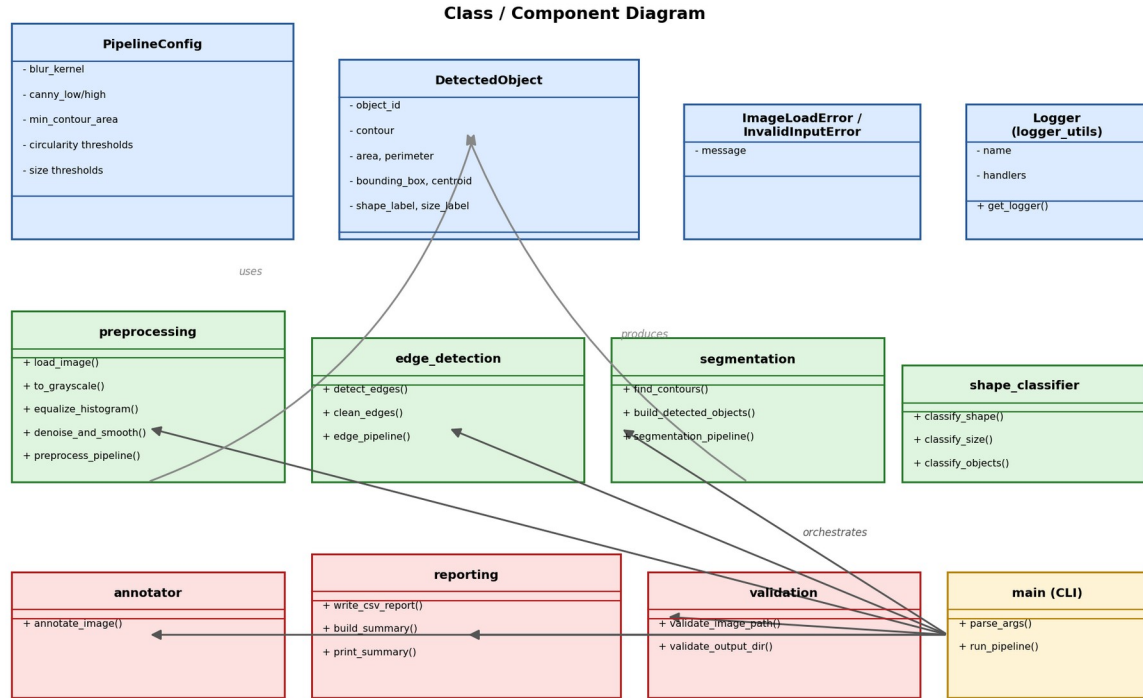


Figure 7.4 — Class / Component Diagram

7.5 Database / Storage Design (ER Diagram)

This project does not use a relational database. All persistent output is written as flat files (an annotated PNG image and a CSV report), so a traditional Entity-Relationship diagram is not applicable. For completeness, the schema of the CSV output (which plays the role of the project's only persisted “table”) is documented below.

Column	Type	Description
object_id	Integer	Unique sequential ID assigned to each detected object
shape_label	String	circle / triangle / square / rectangle / polygon / unknown
size_label	String	small / medium / large
area_px2	Float	Contour area in square pixels
perimeter_px	Float	Contour perimeter in pixels
centroid_x, centroid_y	Integer	Pixel coordinates of the object's centroid
bbox_x, bbox_y, bbox_w, bbox_h	Integer	Axis-aligned bounding box of the object
circularity	Float	$4\pi \cdot \text{Area} / \text{Perimeter}^2$ (1.0 = perfect circle)

8. Design Decisions & Rationale

8.1 Classical CV over Machine Learning

A geometric, rule-based approach was chosen over a trained object detector (e.g., YOLO) because the target scenes (objects on a plain, evenly lit background) do not require the generalization power of deep learning, and a classical pipeline avoids the need for a labeled dataset, training infrastructure, and GPU inference — while remaining fully interpretable, since every classification can be traced back to a specific geometric measurement.

8.2 CLAHE over Global Histogram Equalization

Contrast-Limited Adaptive Histogram Equalization (CLAHE) was selected instead of global equalization (`cv2.equalizeHist`) because CLAHE operates on small tiles of the image and clips the histogram, which prevents it from over-amplifying noise in the large, near-uniform background regions typical of object-counting photographs.

8.3 Canny + Morphological Closing over Simple Thresholding

Simple global or adaptive thresholding was considered but rejected as the primary detection mechanism, because it is highly sensitive to uneven illumination and shadow. Canny edge detection, followed by a morphological closing and dilation pass to bridge small gaps in the edge map, produces more robust and continuous object boundaries suitable for contour extraction, directly reflecting Module 3 of the syllabus (edge-based segmentation).

8.4 Contour Geometry over Hough Transform for Shape ID

The Hough Transform (also covered in Module 3) is powerful for detecting a single known shape (e.g., only circles, or only lines) but becomes considerably more complex when several different shape types must be distinguished simultaneously. Polygon approximation (`cv2.approxPolyDP`) combined with the circularity metric was chosen instead, since a single, consistent computation (vertex count + circularity) cleanly separates circles, triangles, squares/rectangles, and general polygons.

8.5 Single Configuration Object

All tunable numeric thresholds (Canny thresholds, minimum contour area, circularity cutoff, size-bucket boundaries, etc.) are grouped into one `PipelineConfig` dataclass rather than scattered as magic numbers throughout the code. This directly supports the maintainability and configurability non-functional requirements: retuning the system for a new image domain (e.g., larger or smaller objects, different lighting) requires editing values in exactly one place.

8.6 Strict Modular Separation

Each functional concern (pre-processing, edge detection, segmentation, classification, annotation, reporting, validation, logging) lives in its own module with a narrow, well-defined interface. This mirrors standard software-engineering practice, keeps each file small and independently testable (see Section 11), and matches the “minimum 5–10 meaningful modules/files” technical expectation of the project brief.

9. Implementation Details

9.1 Technology Stack

Component	Choice	Reason
Language	Python 3.10+	Rich CV ecosystem, fast prototyping, readable for reporting/grading
Core CV Library	OpenCV (opencv-python-headless)	Industry-standard implementations of Canny, contours, morphology, CLAHE
Numerical Computing	NumPy	Efficient array operations underlying every OpenCV call
Report Visuals	Matplotlib	Used only for generating report figures/diagrams, not part of the runtime pipeline
Testing	unittest (standard library)	No extra dependency; integrates directly with python -m unittest
CLI	argparse (standard library)	Simple, dependency-free command-line interface

9.2 Key Algorithmic Steps

- Grayscale conversion: `cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)`
- Contrast enhancement: `cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))`
- Noise suppression: `cv2.GaussianBlur(image, (5,5), 0)`
- Edge detection: `cv2.Canny(image, 50, 150)`
- Gap bridging: `cv2.morphologyEx(edges, cv2.MORPH_CLOSE, kernel) + cv2.dilate(...)`
- Contour extraction: `cv2.findContours(edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)`
- Noise filtering: contours with area $< 150 \text{ px}^2$ (default) are discarded as background/texture noise
- Shape descriptor: circularity = $4\pi \cdot \text{Area} / \text{Perimeter}^2$; polygon vertex count via `cv2.approxPolyDP` with $\epsilon = 3\%$ of the perimeter
- Size buckets: small $\leq 1500 \text{ px}^2$, medium $\leq 6000 \text{ px}^2$ (default; both configurable), else large

9.3 Configuration Reference (modules/config.py)

```

blur_kernel = (5, 5)           canny_low = 50           canny_high = 150
min_contour_area = 150.0       circularity_circle_min = 0.75
small_max_area = 1500.0       medium_max_area = 6000.0

```

9.4 Command-Line Interface

```

python main.py --image sample_images/coins_sample.png --output output/coins_run
python main.py --image <path> --output <dir> --min-area 300 --canny-low 40 --
canny-high 130

```

9.5 Error Handling & Logging in Practice

If the supplied image path does not exist, has an unsupported extension, or cannot be decoded by OpenCV, the pipeline raises a typed exception (`InvalidInputError` or `ImageLoadError`), logs the failure to `output/pipeline.log`, prints a clear one-line error to the console, and exits with a non-zero status code instead of crashing with a raw Python traceback.

10. Screenshots / Results

10.1 Pre-processing Stages (Coins Sample)

The figure below shows every intermediate stage of Module 1 and the edge-detection half of Module 2, run on the synthetic coins sample image.

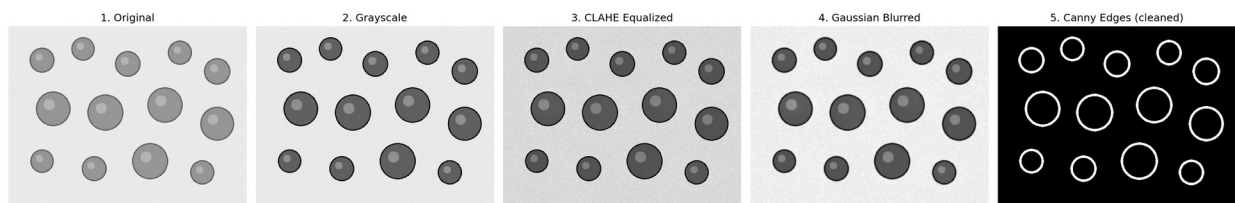


Figure 10.1 — Original → Grayscale → CLAHE → Gaussian Blur → Canny Edges

10.2 Result 1 — Mixed Geometric Shapes

Input: `sample_images/shapes_sample.png` (a synthetic image containing circles, squares, rectangles, triangles, and a pentagon of varying sizes).

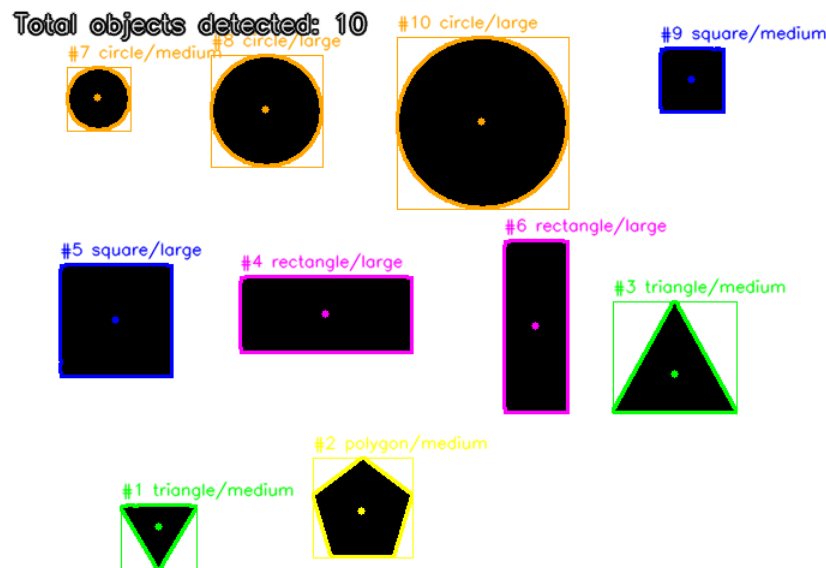


Figure 10.2 — Annotated Output: 10/10 objects correctly detected and classified

Shape	Count	Size	Count
circle	3	medium	5

Shape	Count	Size	Count
square	2	large	5
rectangle	2		
triangle	2		
polygon	1		
Total	10	Total	10

10.3 Result 2 — Simulated Coins

Input: sample_images/coins_sample.png (13 shaded, circular “coins” of two denominations scattered on a textured background).

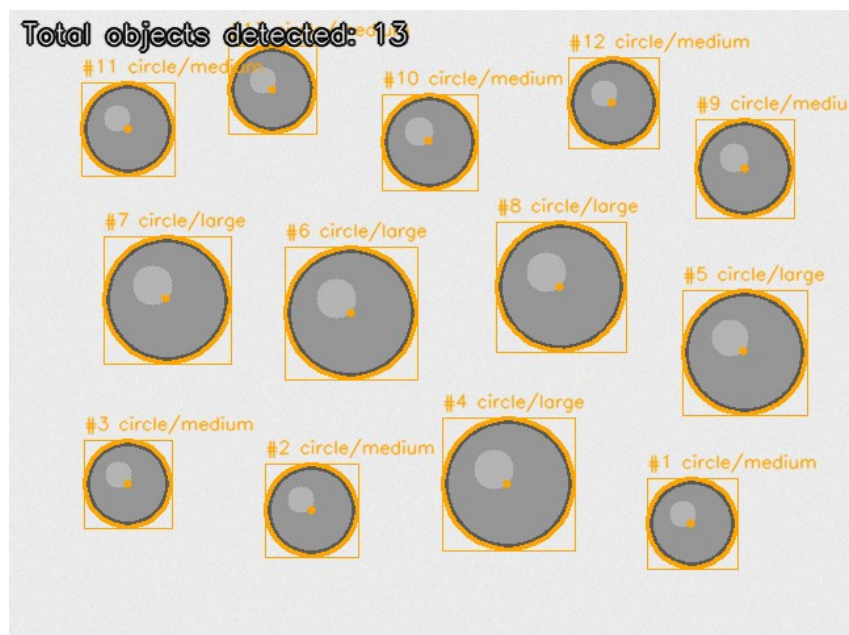


Figure 10.3 — Annotated Output: all 13 coins correctly detected as circles

Shape	Count	Size	Count
circle	13	medium	8
		large	5
Total	13	Total	13

10.4 CSV Report Excerpt (coins_run/detection_report.csv)

id	shape	size	area_px2	perimeter_px	circularity
1	circle	medium	3577.0	225.14	0.887
2	circle	medium	3790.0	230.79	0.894

id	shape	size	area_px2	perimeter_px	circularity
3	circle	medium	3352.0	217.14	0.893
4	circle	large	7736.0	329.71	0.894
5	circle	large	6846.0	310.39	0.893

11. Testing Approach

The project uses Python's built-in unittest framework. Every functional module has a dedicated test file that exercises it in isolation using small, synthetically generated images and objects, so the test suite is fully deterministic and requires no external files or network access.

Test File	Module Under Test	# Tests	What is verified
test_preprocessing.py	preprocessing.py	6	Image loading (success/failure), grayscale conversion, CLAHE output shape/type, blur reduces variance, full pipeline keys
test_edge_detection.py	edge_detection.py	4	Canny finds real boundaries, morphology output is binary, pipeline keys present, blank image → no edges
test_segmentation.py	segmentation.py	4	Correct contour count, descriptor computation, area-filter excludes objects, end-to-end object IDs
test_shape_classifier.py	shape_classifier.py	5	Every object gets a label, circle/triangle/square are correctly identified, size-bucket boundaries
test_validation_and_reporting.py	validation.py, reporting.py	6	Invalid path/extension rejected, positive-number check, output dir auto-created, CSV row count, summary counts

Command to run the full suite:

```
python -m unittest discover -s tests -v
```

Result obtained on the final build:

```
Ran 25 tests in 0.088s
```

```
OK
```

All 25 tests pass. In addition to unit tests, the pipeline was validated end-to-end (integration testing) by running main.py against both sample images and manually verifying that the detected counts (10/10 shapes, 13/13 coins) and shape/size labels exactly matched the known ground truth used to generate the synthetic images.

12. Challenges Faced

- Broken edge contours: Canny edges were occasionally not fully closed around an object, which caused `cv2.findContours` to miss or merge objects. This was resolved by adding a morphological closing + dilation step before contour extraction.
- Distinguishing squares from rectangles at small sizes: at very small pixel sizes, polygon approximation sometimes produced 4 vertices that were ambiguous between a square and a narrow rectangle; an aspect-ratio check (bounding-box width/height close to 1.0) was added as a tie-breaker.
- Circularity threshold tuning: near-circular polygons (e.g., a regular hexagon or the pentagon in the shapes sample) could be misclassified as circles if the circularity threshold was too permissive; the combination of a circularity cutoff AND a minimum vertex count (≥ 6) resolved this.
- Background noise contours: fine texture noise on the simulated coin background produced many tiny spurious contours; a minimum-area filter (configurable via `--min-area`) was introduced to discard them.
- Generalizing thresholds across images: the same Canny thresholds do not work equally well on every image; exposing `--canny-low`, `--canny-high`, and `--min-area` as CLI overrides (on top of the config defaults) allows quick re-tuning without editing code.

13. Learnings & Key Takeaways

- Classical CV pipelines can solve real, practical problems (counting/sorting) with no training data, and remain fully explainable — an important trade-off compared to deep learning.
- Pre-processing quality (contrast enhancement, noise suppression) has a direct, often underestimated, impact on the quality of downstream edge detection and segmentation.
- Simple geometric descriptors (circularity, vertex count, aspect ratio) are surprisingly effective for shape classification when the scene is well-controlled.
- Centralizing configuration and logging from the start made iterative threshold tuning and debugging significantly faster than hard-coding values across files.
- Writing unit tests alongside each module (rather than only at the end) caught several edge cases early — particularly the area-filter boundary conditions and the blank-image edge case.

14. Future Enhancements

- Watershed segmentation or distance-transform-based splitting to correctly separate touching/overlapping objects, which the current contour-only approach cannot do.
- Adaptive/automatic Canny threshold selection (e.g., based on the median pixel intensity) to reduce the need for manual `--canny-low/--canny-high` tuning per image.
- Batch mode to process an entire folder of images in one run and aggregate statistics across the whole batch.
- A simple web-based front-end (e.g., Streamlit) for drag-and-drop image upload and interactive threshold sliders.

- Optional deep-learning-based detector (e.g., a lightweight YOLO model) as an alternate backend for cluttered or unevenly lit scenes where classical edge detection struggles, with the current pipeline retained as a fast, explainable baseline.
- Real coin/currency value estimation by calibrating pixel-to-millimeter scale and matching detected coin diameters to known denominations.

15. References

- [1] R. Szeliski, Computer Vision: Algorithms and Applications, Springer-Verlag London, 2011.
- [2] R. C. Gonzalez and R. E. Woods, Digital Image Processing, Addison-Wesley, 1992.
- [3] D. A. Forsyth and J. Ponce, Computer Vision: A Modern Approach, Pearson Education, 2003.
- [4] OpenCV Developers, “OpenCV Documentation — Image Processing and Contours,” <https://docs.opencv.org/>
- [5] CSE3010 Computer Vision Course Syllabus (LP), VIT, compiled by Dr. Soundarrajan, approved 20.01.2020.
- [6] VITyarthi — Build Your Own Project: General Project Instructions & Submission Guidelines.